

Data Analysis 1

Ryan Womack

2026-09-09

Copyright Ryan Womack, 2026. This work is licensed under [CC BY-NC-SA 4.0](#)

Data Analysis 1

overview, data methods, and data wrangling with R + comparison with Python

1 Overview

This workshop reviews the basics of importing, examining, and manipulating data (“data wrangling”) in R, with some brief comparisons to the Python approach to accomplishing the same tasks.

1.1 Installation

This workshop does *not* cover the installation of R and RStudio, but assumes that you have a working copy available.

R is open source software for statistical analysis, available at [R-project.org](#). It is recommended to install R first. R commands can be executed by many editors and IDEs, directly in the terminal, or via script files. However, one of the most powerful and common methods, which we will use here, is to use the RStudio IDE.

RStudio, available from [Posit](#) as an [open source download](#), is a tool for creating and running R code (and Python and a few other languages), with many helpful features and templates for working with the other formats and tools in the R ecosystem. RStudio will make use of your installed version of R, running as a layer on top. Posit is also developing [Positron](#) as a newer IDE with more flexibility to work with R, Python, and other possible languages. This workshop does not use Positron, although the code should also work there.

For immediate use without installation, try the following:

- R/RStudio is also available via [Rutgers OIT Computer Labs](#) and their [Virtual Lab](#).
- The [Posit Cloud](#) is also a free service that allows you to use RStudio (with some limitations on time and memory) with the creation of an account.

Please see the [Getting Started with R video](#) for more details (or other web tutorials and guides).

[Python](#) is also open-source, and can be used with various editors and tools, which are not detailed here.

2 Setup and preparing data

2.1 R packages required

We will use the [pak](#) package for installation as a more complete approach to package management. Replace the `pkg` commands with `install.packages()` versions if you prefer.

This session relies on the [tidyverse](#) suite of packages for data manipulation as a preference, although the same tasks could be accomplished in base R using other functions. Install *pak* and *tidyverse* if you don't already have them on your system. We will also need *reticulate* to run Python code chunks.

```
install.packages("pak", dependencies=TRUE)
library(pak)
pkg_install("tidyverse")
pkg_install("reticulate")
devtools::session_info()
```

Note that the `update.packages()` can be run periodically to check for newer versions of any packages installed.

Now let's load the tidyverse and reticulate (for Python support only)

```
library(tidyverse)
```

```
-- Attaching core tidyverse packages ----- tidyverse 2.0.0 --
v dplyr      1.2.1      v readr      2.2.0
v forcats    1.0.1      v stringr    1.6.0
v ggplot2     4.0.3      v tibble     3.3.1
v lubridate  1.9.5      v tidyr      1.3.2
v purrr       1.2.2
```

```
-- Conflicts ----- tidyverse_conflicts() --
x dplyr::filter() masks stats::filter()
x dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become
```

```
# Sys.setenv(RETICULATE_PYTHON = "/usr/bin/python3")
library(reticulate)
use_python("/usr/bin/python3")
```

3 Basics of the R language

3.1 Case sensitivity

R is case sensitive. While `Getwd()` will return an error when typed in the console, `getwd()` will show you the current working directory. On the other hand, `getwd` without parentheses will print the internal workings of the command. R is often terse and offers little feedback. A successful command may simply return you to a blank cursor, while an error may require some searching on [Stack Exchange](#) or similar sites to discover how to fix it.

3.2 Console as calculator

The console may be used as a calculator, providing answers for everything from $2+2$ to 2^{1000} (2 to the 1000th power). $2+2$. However, 2^{1024} will exceed the range of numbers that R can calculate, returning “Inf” for infinity. Most other functions such as $\ln$ or e that one would find on a scientific calculator are available.

3.3 Sampling and probability

R is built for statistics and has a full range of sampling capabilities and probability distributions. Parameters can be passed to customize these functions. See the [CRAN Task View for Probability Distributions](#) for more details.

```
# draw ten numbers at random from the range 1 to 100
```

```
sample(1:100,10)
```

```
[1] 91 99 35 68 61 50 14 20 88 9
```

```
# draw ten numbers from the standard normal distribution
```

```
rnorm(10)
```

```
[1] 2.2174796 1.0074451 1.2413267 1.2391878 -0.1923956 -0.2751464  
[7] -1.4888921 -0.6985675 -1.9855195 0.7884238
```

```
# draw ten numbers from a normal distribution with mean 100 and standard  
↪ deviation 20
```

```
rnorm(10, mean=100, sd=20)
```

```
[1] 77.62646 117.77151 91.29705 99.55333 145.80074 87.61931 62.74401  
[8] 102.90501 116.98587 83.45364
```

We can also sample from a data object, such as a list of respondent names. In addition to sampling from distributions, we can calculate values based on the probability distribution functions (PDF), cumulative distribution functions (CDF), and quantiles of these distributions in a very flexible manner. See the [Introduction to R manual](#) for a brief discussion. There are also many other tutorials on the Web.

3.4 Creating functions

Creating your own functions is easy. Name the function, use the assignment operator (`<-`), then the *function* argument, enclosing the actual workings of the function in curly brackets `{}`. While the example below is simple, complex conditional statements of any length can be used within the brackets.

```
# define the function

funkyadd<-function(x,y)
{
  x+y+1
}

# run the function with arguments

funkyadd(2,2)

[1] 5
```

The ease of adding functions is one element in the flourishing of the large (22,000+) ecosystem of R packages, as individual researchers and users build the functions and tools to meet their analytical needs.

3.5 R help

The R help system is easy to access. Use a question mark before the name of a specific function to pull up its manual page (`?sample` or `?rnorm`). Use two question marks to search The help tab in the lower right quadrant of RStudio also allows full access to the built in help, including manuals and searching.

Searching the internet for R help can be useful, since there are a wealth of tutorials, blog posts, and other material on general methods and specific R packages. It is helpful to add something like “R statistics” or “R software” or “R package” to your search to distinguish from all other uses of the letter R!

Two subscription resources available to Rutgers students are the [O’Reilly](#) collection of technology resources, including books, videos, and mini-courses, and [Springer Nature](#), which has a series called “Use R!” that provides practical guides to both general and specialized disciplinary methods for working with R. Just search for “Use R!” once logged in.

3.6 RStudio

Here we also briefly review the basic layout of RStudio in the live demo or video.

4 The *tidyverse*

We will be preferring [tidyverse](#) commands and methods throughout this workshop. The tidyverse is a very functional and well-structured collection of packages, backed by Posit, that is an excellent entry point into the world of R. However, one should be aware that as with most things in R, there are multiple possible paths to accomplishing the same task. So do not feel limited to this approach.

5 Importing data

5.1 readr

readr is the basic tidyverse package for importing plain text files of data into R.

start with a tab-separated file

```
download.file("https://ryanwomack.com/data/myfile.txt", "myfile.txt")
mydata <- read_tsv("myfile.txt")
```

Rows: 4 Columns: 3

-- Column specification -----

Delimiter: "\t"

chr (1): sex

dbl (2): age, height

i Use `spec()` to retrieve the full column specification for this data.

i Specify the column types or set `show\_col\_types = FALSE` to quiet this message.

mydata

A tibble: 4 x 3

	sex	age	height
	<chr>	<dbl>	<dbl>
1	M	22	170
2	F	33	180
3	M	44	190
4	F	55	200

`read_tsv` imports tab-separated text files. Later we will see `read_csv` which imports the most widely used comma-separated text file format. Among data formats, this is highly interoperable and error-proof, and is recommended for data sharing wherever appropriate.

Instead of the underscore (`_`) typical of *tidyverse* data import commands, base R will use a period (`.`) separator, as in the `read.csv` command.

See readr.tidyverse.org for complete details. While the defaults for these functions are reasonable, the options outlined there will help you to adjust the import process for your particular use case and to deal with any errors.

As described on their site, the *tidyverse* is opinionated. In the case of data import, the *tidyverse* style commands will make fewer assumptions about data you are imported. If an inconsistency is encountered, it is more likely to throw an error and ask you to fix it, rather than silently choosing an option to resolve it. In base R data import, this results in the data import appearing to be “easy”, and then oddities later in the analysis appearing, resulting from assumptions like coercing all data to be a certain type. In the *tidyverse*, the initial data import may require more patience and tweaking, but the resulting data frame will behave exactly as you have chosen.

Note that for any data we can import, we can also export in that format. For csv files, use the `write_csv` command.

5.2 Excel import

The [readxl](#) package is used for .xls and .xlsx sheets. One thing to note is the number used to indicate which of the sheets in a file to import. If the sheet is named, that can also be used. The [writexl](#) package exports Excel-format files.

```
# install.packages("readxl")
library(readxl)

download.file("https://ryanwomack.com/data/mydata.xlsx", "mydata.xlsx")
mydata<-read_excel("mydata.xlsx", 1)
mydata

# A tibble: 4 x 3
  sex    age height
<chr> <dbl> <dbl>
1 M      22    170
2 F      33    180
3 M      44    190
4 F      55    200
```

There are other non-tidyverse alternatives to working with Excel files that you may encounter.

5.3 Importing other formats

- The [haven](#) package is used to read and write SAS, Stata, and SPSS formats. The [foreign](#) package is the traditional base R way of accomplishing these tasks.
- [googledrive](#) allows you to interact with files on Google Drive from R.
- [rvest](#) for web scraping.

The following are not in the core tidyverse, but are useful nevertheless.

- [jsonlite](#) for JSON files.
- [xml2](#) for working with XML.
- [httr](#) for interacting with web APIs.
- [feather](#) for sharing with Python and other languages.
- [DBI](#) for working with relational databases. To connect to a specific database, you'll need to pair DBI with a specific backend like RSQLite, RPostgres, or odbc. Learn more at <https://solutions.posit.co/connections/db/>.
- [Rclone](#) for working with cloud sites like AWS, Box, Dropbox, and many more. Note that Rclone is not an R package (in spite of its name), but a standalone command line application. However, *rclone* commands can be run from with R or RStudio.

There are even more things that can be done with data, since R has packages for nearly everything, but let's move on to working with data, a.k.a. "data wrangling".

6 Wrangling your data

6.1 tibbles and data frames

To quote the [tibble](#) webpage, "tibbles are data.frames that are lazy and surly: they do less (i.e. they don't change variable names or types, and don't do partial matching) and complain more (e.g. when a variable does not exist). This forces you to confront problems earlier, typically leading to cleaner, more expressive code. Tibbles also have an enhanced `print()` method which makes them easier to use with large datasets containing complex objects."

While the tibble is a "better-behaved" data.frame, at least in some ways *data.frame* is the traditional R data structure and many packages still expect it. We can convert back and forth between tibbles and data frames easily, however. The *as.data.frame* command converts a tibble to a data frame, while the *as_tibble* command converts a data frame to a tibble.

```
library(tibble)
```

```
# "as" functions in R convert back and forth between formats
```

```
# "." notation in base R, "_" notation in tidyverse
```

```
as.data.frame(mydata)
```

```
sex age height
1  M  22    170
2  F  33    180
3  M  44    190
4  F  55    200
```

```
as_tibble(iris)
```

```
# A tibble: 150 x 5
  Sepal.Length Sepal.Width Petal.Length Petal.Width Species
      <dbl>         <dbl>         <dbl>         <dbl> <fct>
1         5.1         3.5         1.4         0.2 setosa
2         4.9         3         1.4         0.2 setosa
3         4.7         3.2         1.3         0.2 setosa
4         4.6         3.1         1.5         0.2 setosa
5         5         3.6         1.4         0.2 setosa
6         5.4         3.9         1.7         0.4 setosa
7         4.6         3.4         1.4         0.3 setosa
8         5         3.4         1.5         0.2 setosa
9         4.4         2.9         1.4         0.2 setosa
10        4.9         3.1         1.5         0.1 setosa
# i 140 more rows
```

6.2 data.table

The [data.table](#) package is not a part of the tidyverse, but is important to be aware of. It is a high performance version of the data frame suitable for big data applications, using the *fread* command to import data.

6.3 World Bank Gender Statistics

Now let's grab some data. We will use a realistic data example, the [World Bank's Gender Statistics database](#), whose [raw data](#) is directly downloadable. Other [World Bank Open Data](#) is available as well. See genderdata.worldbank.org for more background on the Gender Data portal. Note that we have to inspect the data and understand the variables first before manipulating in R – this is not an automatic process. The dataset we download is over 150MB in size.

Our hypothetical researcher wants to isolate selected variables for selected countries for one recent year of data. We'll work through this step by step to illustrate data commands at our disposal for wrangling.

```
# note the timeout options below may be useful if you're on a slow connection
↳ or the World Bank site is slow to respond.
```

```
getOption("timeout")
```

```
[1] 60
```



```

options(timeout=6000)

download.file("https://databank.worldbank.org/data/download/Gender_Stats_CSV.zip",
  ↪ "gender.zip")

unzip("gender.zip")

# we read in the data with read_csv and examine it, then remove two unneeded
↪ columns with codes, using basic matrix notation

gender_data <- read_csv("Gender_StatsCSV.csv")

Rows: 372504 Columns: 70
-- Column specification -----
Delimiter: ","
chr (4): Country Name, Country Code, Indicator Name, Indicator Code
dbl (66): 1960, 1961, 1962, 1963, 1964, 1965, 1966, 1967, 1968, 1969, 1970, ...

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

names(gender_data)

[1] "Country Name" "Country Code" "Indicator Name" "Indicator Code"
[5] "1960"         "1961"         "1962"         "1963"
[9] "1964"         "1965"         "1966"         "1967"
[13] "1968"         "1969"         "1970"         "1971"
[17] "1972"         "1973"         "1974"         "1975"
[21] "1976"         "1977"         "1978"         "1979"
[25] "1980"         "1981"         "1982"         "1983"
[29] "1984"         "1985"         "1986"         "1987"
[33] "1988"         "1989"         "1990"         "1991"
[37] "1992"         "1993"         "1994"         "1995"
[41] "1996"         "1997"         "1998"         "1999"
[45] "2000"         "2001"         "2002"         "2003"
[49] "2004"         "2005"         "2006"         "2007"
[53] "2008"         "2009"         "2010"         "2011"
[57] "2012"         "2013"         "2014"         "2015"
[61] "2016"         "2017"         "2018"         "2019"
[65] "2020"         "2021"         "2022"         "2023"
[69] "2024"         "2025"

gender_data <- gender_data[,c(-2,-4)]
names(gender_data)

[1] "Country Name" "Indicator Name" "1960"         "1961"
[5] "1962"         "1963"         "1964"         "1965"
[9] "1966"         "1967"         "1968"         "1969"

```

[13]	"1970"	"1971"	"1972"	"1973"
[17]	"1974"	"1975"	"1976"	"1977"
[21]	"1978"	"1979"	"1980"	"1981"
[25]	"1982"	"1983"	"1984"	"1985"
[29]	"1986"	"1987"	"1988"	"1989"
[33]	"1990"	"1991"	"1992"	"1993"
[37]	"1994"	"1995"	"1996"	"1997"
[41]	"1998"	"1999"	"2000"	"2001"
[45]	"2002"	"2003"	"2004"	"2005"
[49]	"2006"	"2007"	"2008"	"2009"
[53]	"2010"	"2011"	"2012"	"2013"
[57]	"2014"	"2015"	"2016"	"2017"
[61]	"2018"	"2019"	"2020"	"2021"
[65]	"2022"	"2023"	"2024"	"2025"

6.4 tidyr

The goal of [tidyr](#) is to help you create tidy data. Tidy data is data where:

- Every column is a variable.
- Every row is an observation.
- Every cell is a single value.

First we use *pivot_longer* to create long format data. [Long vs. Wide data explained](#). Then we filter the data for a recent year (2024 has relatively complete data). The *pivot_wider* to restore the data to wide format.

Note that we are also renaming the datasets as we go. This lets us easily go back to an earlier version if we mess anything up. As long as you have enough memory to do this, it is highly recommended.

```
# pivot_longer
```

```
gender_data2 <-
  gender_data |>
  pivot_longer(3:68, names_to = "Year", values_to = "Value")
```

```
# filter
```

```
gender_data2024 <-
  gender_data2 |>
  filter(Year=="2024")
```

```
# we no longer need the Year variable, since all are 2024
```

```
gender_data2024 <- gender_data2024[, -3]
```

```
# pivot_wider
```

```
gender_data2024wide <-
  gender_data2024 |>
  pivot_wider(names_from = "Indicator Name", values_from = "Value")

# write this version of the data to a file

write_csv(gender_data2024wide, "widedata.csv")
```

In much older R code, you will commonly see the “%>%” characters used to pass commands from one line to the next. This is called the “pipe” and is functionality introduced by the *magrittr* package. Since this was used so much, it has been integrated into the *tidyverse* without the need for an additional package. The “|>” character combination is the newer way of expressing this (although the older way will still work). When you see this, you can think of it as “and then” we pass to the next line. So “gender_data2024 |> pivot_wider” means take the gender_data2024 data “and then” use the pivot_wider command on it. This kind of syntax is very human-readable and easy-to-understand for steps in data wrangling.

6.5 summarise

Now that we have created the dataset, we can get descriptive statistics about our data with either *summary* (base R) or *summarise* (tidyverse), which allows us to be more specific about what we would like to examine.

```
ls()
summary(gender_data)
summary(gender_data2024)
summary(gender_data2024wide)

# summarise is the tidyverse way, from dplyr

gender_data2024wide |>
  summarise_if(is.numeric, mean, na.rm=TRUE)
```

Results of this code chunk are not displayed here because they are too voluminous. Note that *ls()* lists items in the workspace, while *rm()* removes them. Do not execute the command below unless you are sure! It removes all files!

```
rm(list = ls())
```

7 dplyr, the heart of the tidyverse

The tidyverse loads the following packages by default, as we’ve seen [readr](#), [tibble](#), [tidyr](#)

We will look briefly at [purrr](#), [forcats](#), [stringr](#), and [lubridate](#) later.

[ggplot2](#), the primary data visualization tool of the tidyverse, is the subject of a later workshop.

However, the [dplyr](#) package is one of the most useful and commonly used, providing a grammar of data manipulation, with a consistent set of verbs that help you solve the most common data manipulation challenges:

- mutate
- select
- filter
- summarise
- arrange

We will look at these one by one. They are both straightforward and powerful when used with various options.

7.1 mutate

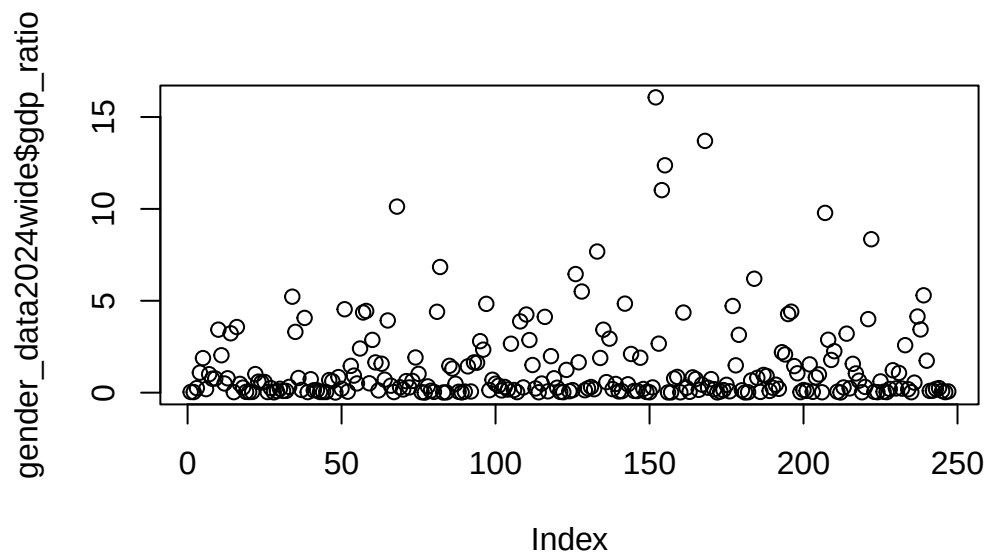
mutate adds new variables that are functions of existing variables

```
gender_data2024wide <-
  gender_data2024wide |>
  mutate(gdp_ratio = (`GDP per capita (Current US$)`/10000)/`Fertility rate`,
         ↪ total (births per woman)`)
```

we can use a function like drop_na to eliminate missing data from a
↪ variable

```
gender_data2024wide <-
  drop_na(gender_data2024wide, gdp_ratio)
```

```
plot(gender_data2024wide$gdp_ratio)
```



```
gender_data2024wide <-
  gender_data2024wide |>
  mutate(hi_ratio = gdp_ratio>0.78)
```

```
attach(gender_data2024wide)
```

7.2 select

`select()` picks variables based on their names. There are several flexible methods for choosing matching names. Here our hypothetical research chooses only the variables beginning with GDP (not case-sensitive).

```
gender_gdp <-  
  select(gender_data2024wide, c(`Country Name`, starts_with("GDP")))  
  
gender_gdp  
  
# A tibble: 247 x 6  
  `Country Name`      `GDP (current US$)` GDP growth (annual %~1  
  <chr>              <dbl>              <dbl>  
1 Africa Eastern and Southern      1.25e12      2.79  
2 Africa Western and Central      7.37e11      4.60  
3 Arab World      3.75e12      2.01  
4 Caribbean small states      9.04e10     13.3  
5 Central Europe and the Baltics    2.46e12      2.10  
6 Early-demographic dividend      1.56e13      3.86  
7 East Asia & Pacific      3.22e13      3.73  
8 East Asia & Pacific (excluding hi~    2.22e13      4.95  
9 East Asia & Pacific (IDA & IBRD)    2.22e13      4.95  
10 Euro area      1.66e13      0.939  
# i 237 more rows  
# i abbreviated name: 1: `GDP growth (annual %)`  
# i 3 more variables: `GDP per capita (constant 2015 US$)` <dbl>,  
#   `GDP per capita (Current US$)` <dbl>, gdp_ratio <dbl>  
  
write_csv(gender_gdp, "gender_gdp.csv")
```

7.3 filter

`filter` picks cases based on their values. Here we further reduce the size of the dataset to include only the GDP variables for a smaller list of 24 countries. We can also filter by criteria.

```
# select countries of interest
```

```
country_list <- c("Armenia", "Azerbaijan", "Canada", "China", "France",  
  ↪ "Georgia", "Germany", "India", "Italy", "Japan", "Kazakhstan", "Korea",  
  ↪ Rep.", "Kyrgyz Republic", "Mexico", "Mongolia", "Russian Federation",  
  ↪ "Saudi Arabia", "Tajikistan", "Turkiye", "Turkmenistan", "United Arab  
  ↪ Emirates", "United Kingdom", "United States", "Uzbekistan")  
  
gender_data2024selected <-  
  gender_gdp |>  
  filter(`Country Name` %in% country_list)
```

```

write_csv(gender_data2024selected, "gender_selected.csv")

# filter by criteria

gender_data2024filtered <-
  gender_data2024selected |>
  filter(gdp_ratio>2)

gender_data2024filtered

# A tibble: 9 x 6
  `Country Name`      `GDP (current US$)` `GDP growth (annual %)`
  <chr>                <dbl>                <dbl>
1 Canada              2.27e12                2.05
2 France              3.16e12                1.19
3 Germany             4.69e12               -0.496
4 Italy               2.38e12                0.783
5 Japan              4.19e12               -0.240
6 Korea, Rep.         1.88e12                2.00
7 United Arab Emirates 5.52e11                3.99
8 United Kingdom      3.70e12                1.08
9 United States       2.93e13                2.79
# i 3 more variables: `GDP per capita (constant 2015 US$)` <dbl>,
#   `GDP per capita (Current US$)` <dbl>, gdp_ratio <dbl>

```

7.4 summarise

summarise reduces multiple values down to a single summary.

```

gender_data2024wide |>
  summarise(mean = mean(gdp_ratio), n = n(), median = median(gdp_ratio))

```

```

# A tibble: 1 x 3
  mean      n median
  <dbl> <int>  <dbl>
1  1.42   247  0.474

```

Usually, you'll want to group first

```

gender_data2024wide |>
  group_by(hi_ratio) |>
  summarise(mean = mean(gdp_ratio, na.rm=TRUE), n = n())

```

```

# A tibble: 2 x 3
  hi_ratio mean      n
  <lgl>    <dbl> <int>
1 FALSE   0.221  151
2 TRUE    3.30   96

```

7.5 arrange

arrange changes the ordering of the rows. It is the dplyr equivalent of *sort* in base R.

```
gender_gdp <-  
  gender_gdp |>  
  arrange(desc(gdp_ratio))
```

```
gender_gdp$gdp_ratio
```

```
[1] 16.070600400 13.701311839 12.371949090 11.022534533 10.124585993  
[6] 9.783150737 8.348997360 7.679928792 6.838878733 6.453597476  
[11] 6.199238174 5.504047944 5.297858233 5.222723959 4.844737956  
[16] 4.831733553 4.717854602 4.540702582 4.448006013 4.407984076  
[21] 4.401256535 4.362591932 4.358775654 4.277524373 4.252443609  
[26] 4.144559986 4.125274435 4.067663828 4.001588116 3.929301220  
[31] 3.877634881 3.572499626 3.439156229 3.430078756 3.426239187  
[36] 3.308385342 3.230273375 3.211524392 3.148558130 2.938878385  
[41] 2.880040766 2.871575448 2.863545595 2.802486413 2.667006960  
[46] 2.663419897 2.583111559 2.402899441 2.340276122 2.256646833  
[51] 2.202067163 2.106135960 2.080021755 2.036222611 1.985979692  
[56] 1.915278677 1.901529545 1.889077415 1.880346602 1.780320195  
[61] 1.740049785 1.652868670 1.647256450 1.646379084 1.630921107  
[66] 1.574777530 1.570160776 1.539332157 1.505022393 1.487433417  
[71] 1.461312743 1.457637994 1.444619397 1.430446348 1.312252342  
[76] 1.239050111 1.219623115 1.088857881 1.073832144 1.055140865  
[81] 1.046473536 1.023076556 1.011905815 1.005491085 0.992677079  
[86] 0.964900516 0.928841999 0.909839451 0.857650728 0.848173645  
[91] 0.834010944 0.832703857 0.808820864 0.799211266 0.783816329  
[96] 0.783214815 0.770890974 0.769567555 0.759425115 0.739325707  
[101] 0.737232358 0.731414058 0.707116643 0.703059377 0.687982964  
[106] 0.663704043 0.653107696 0.638819622 0.629886691 0.626834175  
[111] 0.613507986 0.599532960 0.581949884 0.573931307 0.555311448  
[116] 0.545710785 0.510114878 0.503306710 0.499624428 0.497651235  
[121] 0.489453728 0.486602585 0.477611290 0.474350955 0.456872485  
[126] 0.439102095 0.433908227 0.426995865 0.380071450 0.377991662  
[131] 0.342171100 0.314181772 0.312047941 0.311261492 0.308204845  
[136] 0.291790961 0.285864408 0.284226159 0.283072314 0.283012084  
[141] 0.274120147 0.270025205 0.268950356 0.267243677 0.265023898  
[146] 0.252214648 0.248900261 0.248654900 0.232881156 0.232856561  
[151] 0.230343278 0.229121468 0.221881162 0.211273991 0.210870214  
[156] 0.210416824 0.204355187 0.200865350 0.200496449 0.194851038  
[161] 0.193197228 0.188562879 0.188097570 0.185365550 0.175234487  
[166] 0.170155018 0.155001477 0.150653152 0.143839548 0.141880495  
[171] 0.140200825 0.137962855 0.135703719 0.132109667 0.131339362  
[176] 0.131339362 0.129096845 0.122064852 0.121301034 0.110952004  
[181] 0.103134996 0.097403161 0.097193009 0.097161994 0.096723487  
[186] 0.095738810 0.094807663 0.093177071 0.089912212 0.088609891
```

[191]	0.081657611	0.078489749	0.074466210	0.072346049	0.067941076
[196]	0.067365391	0.064778361	0.064489209	0.060410051	0.055040005
[201]	0.053898950	0.050703103	0.048609906	0.046181467	0.045503624
[206]	0.044862503	0.043185964	0.042947861	0.041710672	0.041030108
[211]	0.039257949	0.039102069	0.038402488	0.037109981	0.036177025
[216]	0.036177025	0.036138135	0.033226186	0.033155808	0.032067120
[221]	0.030717812	0.030603611	0.029413019	0.029079207	0.029006979
[226]	0.027685787	0.026573758	0.026263085	0.025880259	0.024741224
[231]	0.023904942	0.023091174	0.022302029	0.022036691	0.021783814
[236]	0.019841240	0.018112236	0.016268552	0.014550567	0.014068987
[241]	0.014008020	0.012297501	0.011584813	0.010469631	0.008755958
[246]	0.008669147	0.004515179			

```
write_csv(gender_gdp, "gender_gdp.csv")
```

See the [articles page](#) in the dplyr reference for more information about combining and controlling these operations (joining datasets, working across columns or rows of data, and more).

8 Additional wrangling tools

In addition to tidyr and dplyr, there are five tidyverse packages which are designed to work with specific types of data:

- [lubridate](#) for dates and date-times.
- [hms](#) for time-of-day values.
- [blob](#) for storing blob (binary) data.
- [stringr](#) provides a cohesive set of functions designed to make working with strings as easy as possible.
- [forcats](#) provides a suite of useful tools that solve common problems with factors. R uses factors to handle categorical variables.

Each of these provides very convenient functions for handling data that might otherwise be tricky with general methods.

8.1 purrr

The [purrr](#) package enhances R's functional programming (FP) toolkit by providing a complete and consistent set of tools for working with functions and vectors. Once you master the basic concepts, purrr allows you to replace many for loops with code that is easier to write and more expressive.

For example, the “map” family of commands allows you to apply a function to each element of a vector.


```
# an example of map in action
```

```
mtcars |>
  split(mtcars$cyl) |>
  map(\(df) lm(mpg ~ wt, data = df)) |>
  map(summary) |>
  map_dbl("r.squared")
```

```
           4           6           8
0.5086326 0.4645102 0.4229655
```

8.2 glue

`glue` provides an alternative to base R's `paste()` that makes it easier to combine data and strings.

9 broom

`broom` turns models into tidy data which you can then wrangle and visualise using the tools you already know. The key commands are *tidy*, *glance*, and *augment*

```
library(broom)
```

```
regoutput<-lm(`GDP per capita (constant 2015 US$)`~`Fertility rate, total
↪ (births per woman)`, gender_data2024wide)
```

```
# base R regression summary
summary(regoutput)
```

Call:

```
lm(formula = `GDP per capita (constant 2015 US$)` ~ `Fertility rate, total (births per woman)`,
    data = gender_data2024wide)
```

Residuals:

```
      Min       1Q   Median       3Q      Max
-27040 -12878  -5256   4019 228048
```

Coefficients:

	Estimate	Std. Error	t value	Pr(> t)
(Intercept)	38262	3312	11.554	< 2e-
16				
`Fertility rate, total (births per woman)`	-9105	1243	-7.327	3.46e-
12				

```
(Intercept) ***
`Fertility rate, total (births per woman)` ***
```

Signif. codes: 0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1

Residual standard error: 23020 on 243 degrees of freedom

(2 observations deleted due to missingness)

Multiple R-squared: 0.181, Adjusted R-squared: 0.1776

F-statistic: 53.69 on 1 and 243 DF, p-value: 3.461e-12

broom variants

`tidy(regoutput)`

A tibble: 2 x 5

	term <chr>	estimate <dbl>	std.error <dbl>	statistic <dbl>	p.value <dbl>
1	(Intercept)	38262.	3312.	11.6	6.72e-25
2	`Fertility rate, total (births per woma~	-9105.	1243.	-7.33	3.46e-12

`glance(regoutput)`

A tibble: 1 x 12

	r.squared <dbl>	adj.r.squared <dbl>	sigma <dbl>	statistic <dbl>	p.value <dbl>	df <dbl>	logLik <dbl>	AIC <dbl>	BIC <dbl>
1	0.181	0.178	23024.	53.7	3.46e-12	1	-2807.	5621.	5631.

i 3 more variables: deviance <dbl>, df.residual <int>, nobs <int>

`augment(regoutput)`

A tibble: 245 x 9

	.rownames <chr>	GDP per capita (constant 2~1 <dbl>	Fertility rate, tota~2 <dbl>	.fitted <dbl>	.resid <dbl>
1	1	1433.	4.16	347.	1086.
2	2	1820.	4.42	-1951.	3771.
3	3	6311.	3.06	10423.	-
4	4	18643.	1.83	21601.	-
5	5	16863.	1.31	26342.	-
6	6	3884.	2.28	17500.	-
7	7	13295.	1.34	26053.	-
8	8	10128.	1.37	25803.	-
9	9	10242.	1.36	25850.	-

```

10 10                                37748.                                1.35  25938.  11809.
# i 235 more rows
# i abbreviated names: 1: `GDP per capita (constant 2015 US$)`,
#   2: `Fertility rate, total (births per woman)`
# i 4 more variables: .hat <dbl>, .sigma <dbl>, .cooks_d <dbl>, .std.resid <dbl>

```

a grouped example

```

regressions <- gender_data2024wide |>
  group_by(hi_ratio) |>
  nest() |>
  mutate(
    fit = map(data, ~ lm(`GDP per capita (constant 2015 US$)`~`Fertility
      ↪ rate, total (births per woman)`, data=.x,)),
    tidied = map(fit, tidy),
    glanced = map(fit, glance),
    augmented = map(fit, augment)
  )

```

```

regressions |>
  unnest(tidied)

```

```

# A tibble: 4 x 10
# Groups:   hi_ratio [2]
  hi_ratio data      fit      term      estimate std.error statistic  p.value glanced
  <lgl>    <list>    <list> <chr>      <dbl>      <dbl>      <dbl>    <dbl> <list>
1 FALSE   <tibble> <lm>    (Inte~    8951.      508.      17.6    2.96e-38 <tibble>
2 FALSE   <tibble> <lm>    `Fert~   -1690.     160.     -10.6    7.60e-20 <tibble>
3 TRUE    <tibble> <lm>    (Inte~   30537.   14418.     2.12    3.69e- 2 <tibble>
4 TRUE    <tibble> <lm>    `Fert~    4208.    9554.     0.440  6.61e- 1 <tibble>
# i 1 more variable: augmented <list>

```

```

regressions |>
  unnest(glanced)

```

```

# A tibble: 2 x 17
# Groups:   hi_ratio [2]
  hi_ratio data      fit      tidied      r.squared adj.r.squared  sigma statistic
  <lgl>    <list>    <list> <list>      <dbl>      <dbl>    <dbl>    <dbl>
1 FALSE   <tibble> <lm>    <tibble>    0.429      0.425    2280.    112.
2 TRUE    <tibble> <lm>    <tibble>    0.00210    -0.00874 31882.     0.194
# i 9 more variables: p.value <dbl>, df <dbl>, logLik <dbl>, AIC <dbl>,
#   BIC <dbl>, deviance <dbl>, df.residual <int>, nobs <int>, augmented <list>

```

```

regressions |>
  unnest(augmented)

```

```

# A tibble: 245 x 14

```

```
# Groups:   hi_ratio [2]
  hi_ratio data      fit    tidied      glanced GDP per capita (constant~1
    <lg1>   <list>   <list> <list>      <list>      <dbl>
1 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>      1433.
2 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>      1820.
3 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>      6311.
4 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>      3884.
5 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>     10128.
6 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>     10242.
7 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>      8719.
8 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>      1016.
9 FALSE   <tibble> <lm>   <tibble [2 x 5]> <tibble>      7516.
10 FALSE  <tibble> <lm>   <tibble [2 x 5]> <tibble>      5826.
# i 235 more rows
# i abbreviated name: 1: `GDP per capita (constant 2015 US$)`
# i 8 more variables: `Fertility rate, total (births per woman)` <dbl>,
#   .fitted <dbl>, .resid <dbl>, .hat <dbl>, .sigma <dbl>, .cooksd <dbl>,
#   .std.resid <dbl>, .rownames <chr>
```

10 Using Python in RStudio, regression example

We will be giving a flavor of the Python approach to these problems, without going in depth. Python is covered in other workshops in greater detail.

Running Python in RStudio requires the *reticulate* package. Your Python installation should also have *numpy* and *scipy* installed. Python will not have direct access to the R data structures, so we use a simplified example here from [Builtin.com](#)

How to pass data from R to Python and vice-versa is described in this [blog post by Dima Diachkov](#)

Note that our code chunk is labeled Python. For more *reticulate* package, see [this post](#)

Also note that it is also possible to invoke R from a Python environment using [rpy2](#)

We will give a quick look at regression in Python, via the [scikit-learn package](#) which provides Linear Regression and many other machine learning techniques. The [statsmodels](#) package provides detailed regression output.

```
import numpy as np
import pandas as pd
from scipy import stats
from sklearn.linear_model import LinearRegression
import statsmodels.formula.api as smf

# import from R
gender_python = r.gender_data2024wide

# print(gender_python)
```

```
# select variables
```

```
labor = gender_python.loc[:, "Labor force participation rate, female (% of  
↳ female population ages 15-64) (modeled ILO estimate)"]  
GDP = gender_python.loc[:, "GDP per capita (Current US$)"]
```

```
# build data frame
```

```
model = LinearRegression()  
df = pd.DataFrame({'labor': labor, 'GDP': GDP})
```

```
# remove missing values
```

```
df_drop = df.dropna()  
print(df_drop)
```

	labor	GDP
0	65.425366	1628.227289
1	68.625080	1416.228412
2	20.345322	7602.411157
3	58.630901	19923.574591
4	68.620596	24616.095787
...	...	...
241	55.458000	3959.877035
242	41.712000	4244.457231
243	75.238000	4716.659938
245	56.678000	1187.109434
246	62.903000	2496.155150

```
[221 rows x 2 columns]
```

```
X = df_drop[['GDP']]  
y = df_drop['labor']  
model = LinearRegression().fit(X, y)  
print(f"Coefficients: {model.coef_}, Intercept: {model.intercept_}")
```

```
Coefficients: [0.00029024], Intercept: 53.00321526309731
```

```
# from statsmodels
```

```
model2 = smf.ols(formula='labor ~ GDP',  
                  data=df).fit()  
print(model2.summary())
```

OLS Regression Results			
=====			
Dep. Variable:	labor	R-squared:	0.172
Model:	OLS	Adj. R-squared:	0.168

```

Method:                Least Squares      F-statistic:           45.55
Date:                  Wed, 09 Sep 2026    Prob (F-statistic):    1.31e-
10
Time:                  10:41:04           Log-Likelihood:        -
912.65
No. Observations:      221               AIC:                   1829.
Df Residuals:          219               BIC:                   1836.
Df Model:               1
Covariance Type:       nonrobust

```

```

=====
              coef      std err          t      P>|t|      [0.025      0.975]
-----
Intercept    53.0032      1.273      41.652      0.000      50.495      55.511
GDP           0.0003     4.3e-05       6.749      0.000       0.000       0.000
=====
Omnibus:                21.439    Durbin-Watson:           1.918
Prob(Omnibus):           0.000    Jarque-Bera (JB):        24.626
Skew:                   -0.768    Prob(JB):                4.49e-
06
Kurtosis:                3.562    Cond. No.                3.71e+04
=====

```

Notes:

- [1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
- [2] The condition number is large, 3.71e+04. This might indicate that there are strong multicollinearity or other numerical problems.

11 R for Data Science

Consult the [R for Data Science book](#) to learn more about data wrangling. Also check out [tidy-models](#) a collection of packages for modeling and machine learning using tidyverse principles.